Physical Design Engineer Interview Questions & Answers

Basics to Advanced, organized by interview round
Including STA, PV, Synthesis, Power, DFT, CAD and Analog Layout roles

126 questions with answers

1. Round 0 — Online Test / Screening
2. Round 1 — Technical Fundamentals (CMOS, Digital, Basic STA)
3. Round 2 — Physical Design Flow (Core PD Round)
4. Round 3 — STA and Signoff (Deep Dive)
5. Round 4 — Advanced / Project / Scenario Round
6. Round 5 — Scripting and Automation
7. Round 6 — Managerial and HR
8. Related Roles — Focus Areas and Sample Q&A
9. Preparation Tips

Round 0 — Online Test / Screening

Usually a 60–90 minute MCQ or written test: aptitude, digital electronics and CMOS basics. Speed and clean fundamentals matter most here.

Digital Electronics Basics

Q1 Implement XOR, AND and OR using only NAND gates. Then using only 2:1 MUXes.

NAND is universal.

- NOT A = NAND(A,A)
- AND = NOT(NAND(A,B)) → 2 NAND gates
- OR = NAND(NOT A, NOT B) → 3 NAND gates
- XOR = 4 NAND gates: N1 = NAND(A,B); Y = NAND(NAND(A,N1), NAND(B,N1))

With a 2:1 MUX (Y = S ? I1 : I0), connect the select to one input:

- NOT A: S=A, I0=1, I1=0
- AND: S=A, I0=0, I1=B
- OR: S=A, I0=B, I1=1
- XOR: S=A, I0=B, I1=B' (needs one extra MUX as the inverter, so 2 MUXes)

Q2 How many NAND gates are needed for a full adder?

9 NAND gates. A half adder needs 5 (4 for XOR plus one shared for carry). Sum = A⊕B⊕Cin uses two 4-NAND XORs (8 gates), and Cout = AB + Cin(A⊕B) reuses the first NAND inside each XOR, so it takes only 1 more NAND gate. Total is 9.

Q3 Difference between a latch and a flip-flop? Build a D flip-flop from latches.

A latch is level-sensitive: it is transparent while the enable is active and holds when it is inactive. A flip-flop is edge-triggered and samples D only at the clock edge. A positive-edge D-FF is a master–slave pair: the master latch is transparent when CLK=0 and the slave latch is transparent when CLK=1. Data captured by the master just before the rising edge passes to Q through the slave. Latches are smaller and allow time borrowing. Flip-flops are easier to time.

Q4 Design a divide-by-2 and a divide-by-3 clock divider with 50% duty cycle.

Divide-by-2: a D flip-flop with D = Q'. It toggles on every rising edge, so Q has half the frequency and 50% duty cycle.

Divide-by-3 (50%): build a mod-3 counter (states 00→01→10) on the positive edge. Its output is high for 1 of 3 cycles. Delay that output by half a cycle with a negative-edge flop, then OR the two signals. The result is high for 1.5 of every 3 cycles, which gives 50% duty cycle. Interviewers check that you know odd dividers need both clock edges.

Q5 Simplify using a K-map with don't-cares, e.g. F(A,B,C,D) = Σm(1,3,7,11,15) + d(0,2,5).

Treat each don't-care as 1 only when it makes a group larger. For this function the groups give F = CD + A'B' (m0, m2 as don't-cares complete the A'B' quad). General rules: group in powers of

2, use the largest groups possible, wrap around the edges, and cover every 1 at least once.

Q6 Draw an FSM to detect sequence 1011 (Mealy and Moore, overlapping).

Mealy (4 states): S0 (reset), S1 ("1"), S2 ("10"), S3 ("101"). Transitions:

- S0: 1→S1, 0→S0
- S1: 0→S2, 1→S1
- S2: 1→S3, 0→S0
- S3: 1→S1 with output 1 (overlap, since the last "1" starts a new sequence), 0→S2

A Moore machine needs a 5th state S4 ("1011", output = 1). From S4: 1→S1, 0→S2. A Mealy machine uses fewer states and responds one cycle earlier. A Moore output is glitch-free because it depends only on state.

Q7 Setup time vs. hold time in one line each.

Setup time is the minimum time data must be stable before the active clock edge. Hold time is the minimum time data must stay stable after the clock edge. Violating either can cause metastability or wrong capture.

CMOS Basics

Q8 Draw a CMOS inverter, NAND and NOR. Why is NAND preferred over NOR?

Inverter: PMOS from VDD to OUT, NMOS from OUT to GND, gates tied to the input. NAND2: two PMOS in parallel (pull-up) and two NMOS in series (pull-down). NOR2: two PMOS in series and two NMOS in parallel. NAND is preferred because its series stack is NMOS. Electron mobility is about 2–3× hole mobility, so series NMOS cost less than series PMOS. NOR needs very wide series PMOS, which makes it larger, slower and higher in capacitance.

Q9 Explain the inverter VTC and noise margins.

The voltage transfer characteristic plots Vout against Vin. It has five regions, depending on which transistor is off, saturated or linear. VIL and VIH are the input points where the slope is −1. VOL and VOH are the output levels. The noise margins are NML = VIL − VOL and NMH = VOH − VIH. The switching threshold VM is where Vin = Vout. A symmetric inverter (VM = VDD/2) gives equal noise margins.

Q10 Why is PMOS sized 2–3× NMOS?

Hole mobility is roughly 2–3× lower than electron mobility. A wider PMOS matches the pull-up drive current to the pull-down. That gives equal rise and fall delays and a centered switching threshold. In FinFET nodes the ratio is smaller (about 1–1.5×) because of strain engineering, and widths are quantized by fin count.

Q11 Sources of power dissipation?

- Dynamic (switching): $P = \alpha \cdot C \cdot V^2 \cdot f$, which charges and discharges load capacitance. Usually the largest part.
- Short-circuit: during an input transition both PMOS and NMOS conduct briefly. Slow input slews make it worse.
- Leakage (static): subthreshold leakage, gate oxide leakage, junction/GIDL leakage. It grows exponentially with lower Vt and higher temperature.

Round 1 — Technical Fundamentals (CMOS, Digital, Basic STA)

Basic

Q12 Define setup time, hold time, clock-to-Q delay, slack, arrival time and required time.

- Tsetup / Thold: the stability windows before and after the clock edge (from the .lib).
- Tclk→q: delay from the clock edge to a valid Q output of the launch flop.
- Arrival time (AT): when data actually reaches the endpoint, i.e. clock latency + Tclk→q + combinational delay.
- Required time (RT): when data must arrive. For setup: capture clock edge − Tsetup. For hold: capture edge + Thold.
- Slack: setup slack = RT − AT. Hold slack = AT − RT. Positive slack passes; negative slack is a violation.

Q13 Write the setup and hold equations including skew.

Let skew = Tcapture_clk − Tlaunch_clk (positive when the capture clock arrives later).

- Setup: Tclk→q + Tcomb(max) + Tsetup ≤ Tperiod + skew − Tuncertainty
- Hold: Tclk→q + Tcomb(min) ≥ Thold + skew + Tuncertainty(hold)

Setup is checked at the next edge with max delays at the slow corner. Hold is checked at the same edge with min delays at the fast corner.

Q14 What is clock skew? When is it good or bad?

Skew is the difference in clock arrival time between two sequential elements. Positive skew (capture clock later) adds time to the setup budget, which helps setup but hurts hold. Negative skew helps hold but hurts setup. Global skew is the max difference across all sinks. Local skew is the difference between launch–capture pairs that share a timing path, and only local skew affects timing.

Q15 What is metastability and how is it reduced?

When data changes inside the setup/hold window, the flop output can hover at an intermediate voltage for an unbounded time before resolving. You mitigate it with 2- or 3-flop synchronizers on CDC signals, handshake or async FIFOs (gray-code pointers) for multi-bit data, and special high-gain synchronizer cells. MTBF = e^(t_r/τ) / (T0 · f_clk · f_data). Each extra flop stage multiplies MTBF exponentially.

Q16 Propagation delay, transition (slew), rise/fall time?

Propagation delay is measured from the input 50% point to the output 50% point. Transition (slew) is the time for a signal to go from 10% to 90% (or 20–80%/30–70%, as defined in the .lib). Rise time is the low→high transition and fall time is high→low. In NLDM, cell delay is looked up from a 2D table of input slew × output load.

Q17 Static vs. dynamic power; give the formula.

Dynamic power $P = \alpha \cdot C \cdot V^2 \cdot f$ (+ short-circuit). Reduce it by lowering V (the most effective, since it is quadratic), lowering activity (clock gating), lowering C (shorter wires, smaller cells) or lowering f. Static power $P = VDD \cdot Ileak$. Reduce it with HVT cells, power gating and body biasing.

Q18 What are the inputs to PnR?

- Gate-level netlist (.v) from synthesis
- SDC: clocks, I/O delays, exceptions
- Liberty (.lib/.db): timing and power for every PVT corner
- Tech LEF: layers, vias, design rules. Cell/macro LEF: abstracts with pins and blockages
- RC tech files (QRC tech / TLU+ / ITF) for parasitic extraction
- UPF/CPF for power intent, an optional floorplan DEF, IO placement file, and MMMC/view definitions

Q19 What does a .lib file contain?

Operating conditions (PVT), units, wire-load models, and for each cell: area, pins (direction, capacitance, max_transition/max_capacitance), functions, timing arcs (cell_rise/fall, rise/fall_transition as NLDM lookup tables indexed by input slew and output load), setup/hold/recovery/removal constraint tables, internal and leakage power, and optionally CCS/ECSM/LVF data.

Q20 LEF vs DEF?

LEF (Library Exchange Format) describes abstract library data. Tech LEF has layers, pitch, width, spacing and vias. Cell LEF has size, site, pin shapes and obstructions. DEF (Design Exchange Format) describes a specific design: die area, rows, tracks, component placement, pins, nets, special nets (power) and routing. Simply put, LEF says what the cells are and DEF says where they are in this design.

Intermediate

Q21 A path has negative setup slack. List every fix.

- Upsize cells on the path (stronger drive) or swap to lower Vt (HVT→SVT→LVT)
- Insert buffers or repeaters on long wires; split high fanout
- Restructure logic: pin swapping (put the late signal on the fast pin), cloning, logic restructuring
- Move cells closer together, reduce wire length, route on higher (less resistive) metal layers, use NDRs
- Use useful skew: delay the capture clock or advance the launch clock
- Reduce crosstalk on the path (spacing/shielding)
- Re-pipeline or retime at RTL, relax constraints if valid (multicycle/false path), or as a last resort lower frequency

Q22 How do you fix hold violations, and why after CTS?

Fixes: insert delay cells or buffers on the data path (at the capture flop's D pin side so other paths aren't affected), downsize cells, swap to HVT, reduce skew between launch and capture. Hold is fixed after CTS because before CTS clocks are ideal (zero skew). The real skew is only

known after the clock tree is built, and hold slack depends directly on that skew. Fixing earlier wastes area and can break setup.

Q23 Can frequency changes fix setup? Hold?

Setup, yes: a larger period gives the data more time. Hold, no: the hold check compares the launch and capture of the same clock edge, so the period does not appear in the hold equation. A hold violation in silicon is fatal at any frequency, which is why hold closure is mandatory.

Q24 False path vs. multicycle path; write the SDC for a 2-cycle MCP.

A false path is a topologically valid path that is never functionally exercised, or one that crosses asynchronous clocks. Timing is not checked on it. A multicycle path is valid but allowed N cycles.

```
set_multicycle_path 2 -setup -from [get_cells A_reg*] -to [get_cells B_reg*]
set_multicycle_path 1 -hold  -from [get_cells A_reg*] -to [get_cells B_reg*]
```

The hold multiplier of (N−1) moves the hold check back to the original launch edge. Without it, hold is checked one cycle later, which makes it far too pessimistic.

Q25 Generated clock vs. virtual clock?

A generated clock is derived from a master clock inside the design (divider, mux or gating output), declared with create_generated_clock -source ... -divide_by. Its latency includes the master's path. A virtual clock has no source pin in the design. It models an external clock for set_input_delay/set_output_delay, so I/O constraints can be referenced to a clock with its own latency and uncertainty.

Q26 NLDM vs CCS vs ECSM?

NLDM uses a voltage-source model with 2D tables of delay and slew against input slew and load. It is simple but loses accuracy below about 130nm with resistive interconnect. CCS (Synopsys) and ECSM (Cadence) are current-source models. They store output current or voltage waveforms over time, which captures Miller effects and resistive loads accurately. They are needed at advanced nodes, give results within about 2% of SPICE, and support noise/receiver capacitance modeling.

Q27 Body effect and short-channel effects?

Body effect: when the source-to-body voltage Vsb is greater than 0, Vt increases ($Vt = Vt0 + \gamma(\sqrt{(2\varphi F+Vsb)} - \sqrt{(2\varphi F)})$). It slows stacked transistors. Short-channel effects:

- DIBL: the drain voltage lowers the barrier, reducing Vt
- Velocity saturation: current becomes roughly linear in Vgs instead of quadratic
- Vt roll-off, hot-carrier injection and punch-through

Q28 LVT vs SVT vs HVT cells?

LVT has a low threshold voltage. It is the fastest but has the highest leakage (often 5–10× SVT). HVT is the slowest with the lowest leakage. SVT/RVT sits in the middle. The usual flow uses mostly HVT/SVT and swaps LVT onto critical paths only. That is multi-Vt optimization for the leakage–timing tradeoff.

Q29 What is latch-up, and how do tap cells prevent it?

Parasitic PNPN (SCR) structures form between the nwell, p-substrate and diffusions. A current injection can trigger a self-sustaining low-impedance path from VDD to GND, which can destroy the chip. Tap (well-tap) cells tie the nwell to VDD and the substrate to VSS at regular intervals. This lowers the well and substrate resistance so the parasitic BJTs can't turn on. Guard rings are used around I/O. The tap spacing rule comes from the foundry, e.g. every 30–60µm.

Q30 FinFET vs planar; why GAA next?

Planar MOSFETs below about 28/20nm suffer severe short-channel effects and leakage because the gate controls the channel from only one side. FinFET wraps the gate around a vertical fin on three sides, which gives better electrostatic control, lower leakage, steeper subthreshold slope and lower VDD. Its drawbacks are quantized width, more self-heating and higher parasitic capacitance. GAA (nanosheet, 3nm/2nm) surrounds the channel on all four sides for even better control. Sheet width is variable, which brings back sizing flexibility.

Round 2 — Physical Design Flow (Core PD Round)

Floorplanning

Q31 Steps in floorplanning?

- Import design (netlist, LEF, lib, SDC) and define die/core size and aspect ratio
- Create rows and tracks, and place I/O pads/pins
- Place macros (memories, IPs) with halos and blockages
- Define power domains or voltage areas (UPF)
- Add endcap and well-tap cells, and physical-only cells
- Build the power plan (rings, straps, rails)
- Run sanity checks: check_design, timing with ideal clocks, early congestion estimate

Q32 Core utilization and aspect ratio?

Core utilization = (standard cell area + macro area) / core area. Typical starting values are 60–75%, leaving room for buffering, CTS and routing. Aspect ratio = height / width of the core. A value of 1 (square) is usually best for routability and power distribution unless the package or IO dictates otherwise.

Q33 Macro placement guidelines?

- Place macros at the periphery and corners so the core stays contiguous for standard cells
- Face macro pins toward the core logic they connect to; follow the data flow (flylines)
- Leave adequate channels between macros for routing and power, and avoid notches and narrow channels
- Group macros of the same hierarchy and align them for regular power-strap access
- Respect orientation rules (some nodes allow only certain orientations, e.g. poly direction)
- Apply halos and place-blockages around macros

Q34 Halo vs blockages (hard, soft, partial)?

A halo (keep-out margin) is an area around a macro where no standard cells may be placed. It moves with the macro. Placement blockages are fixed regions:

- Hard: no cells at all
- Soft: no cells during initial placement, but optimization and CTS buffers are allowed
- Partial: limits the density, e.g. 50%, to ease congestion

Routing blockages prevent routing on specific layers.

Q35 Endcap, tap and decap cells?

Endcap (boundary) cells are placed at the ends of each row. They terminate the well and poly properly to meet DRC and protect the edge cells from process variation. Tap cells connect nwell and substrate to the supplies to prevent latch-up. Decap cells are capacitors between VDD and VSS that supply local charge during switching and reduce dynamic IR drop. They have some leakage cost.

Floorplanning — Advanced

Q36 How do you calculate the channel width between macros?

Channel width ≈ (number of pins crossing the channel × routing pitch) / (number of usable layers × utilization factor), plus room for power straps and halos. Example: 200 signals, 4 usable horizontal layers at 0.1µm pitch, 70% efficiency → 200 × 0.1 / (4 × 0.7) ≈ 7µm, plus PG straps. If cells must sit in the channel, add buffer space. Use a soft or partial blockage in narrow channels to avoid congestion.

Q37 How is pin placement decided in hierarchical designs?

Use the top-level floorplan and flylines. Place pins on the edge facing the connecting partition, align with the neighbour's pins to avoid detours, spread them for routability across allowed layers and pitch, keep timing-critical pins near their logic, and use feedthroughs where blocks abut. Pin assignment is often driven by timing budgets and checked with a global route.

Q38 Flat vs hierarchical flow; budgeting, ILM/ETM?

In a flat flow the whole chip runs as one design. It gives the best QoR but has huge runtime and memory and is hard to parallelize. In a hierarchical flow the chip is split into partitions or blocks that are implemented in parallel by different engineers. It needs timing budgets: the top-level path delay is split into block I/O constraints. Block abstracts used at the top: ILM (Interface Logic Model) keeps only the interface logic and is accurate. ETM (Extracted Timing Model) is a .lib-style timing model and is compact. LEF/FRAM is used for the physical view.

Q39 Why are notches and narrow channels bad?

Cells placed in them get high pin density and few routing tracks, which causes congestion hotspots, detours, DRCs and timing degradation. Power straps also become difficult. Fix them by aligning macros, removing narrow gaps, filling channels with hard or partial blockages, or widening the channels.

Power Planning

Q40 Power rings, straps, rails and grid design?

Rings surround the core or macros on the top metals and collect current from the pads/bumps. Straps (stripes) are a vertical/horizontal mesh on upper layers that distributes current across the core. Rails (followpins) are low-metal (M1/M2) lines along each standard cell row that connect cell VDD/VSS pins. Vias stack from the straps down to the rails. Macros connect through their own power pins/rings. Flip-chip designs use bump-fed grids instead of a peripheral ring.

Q41 How do you decide strap width and pitch?

Estimate total current from power (I = P/V). Choose width and pitch so worst-case IR drop stays within budget (typically less than 5% VDD static, about 10% dynamic) and current density per strap stays below the EM limit (J ≤ Jmax for that layer at the operating temperature). Consider routing resource loss, usually keeping PG under about 15–25% of the top-layer tracks. Then verify with IR analysis and iterate.

Q42 IR drop: static vs dynamic?

IR drop is the voltage loss across the resistive power network (V = I·R), so cells see less than VDD and become slower. Static IR uses average current and catches structural grid weakness. Dynamic IR uses time-varying peak currents during simultaneous switching (vector-based or vectorless). It depends on local switching activity and decap, and it causes timing failures.

Q43 Electromigration and fixes?

EM is metal atom migration caused by momentum transfer from high-density electron flow. Over time it forms voids (opens) or hillocks (shorts). It depends on current density, temperature and wire geometry. Fixes: widen wires or use NDRs, add parallel straps, add more/redundant vias, move to thicker layers, reduce driver strength or fanout, and spread switching. Power nets are DC (unidirectional) EM. Signal nets are AC (bidirectional) EM and are checked with average, RMS and peak limits.

Q44 Power switches, level shifters, isolation cells, retention flops?

- Power switches (header PMOS / footer NMOS) turn a domain's supply off to cut leakage. They are daisy-chained to limit inrush current.
- Level shifters translate signals between different voltage domains (low→high is mandatory; high→low is sometimes needed).
- Isolation cells clamp the outputs of a powered-off domain to a known 0/1 so the active domain doesn't see floating inputs.
- Retention flops have an always-on shadow latch that saves state during power-down for fast wake-up.

Q45 What is UPF/CPF?

UPF (IEEE 1801) and CPF are power intent files kept separate from RTL. They define power domains, supply nets/ports, power switches, isolation, level-shifter and retention strategies, and a power state table (the legal combinations of domain states). The same UPF drives simulation, synthesis, PnR, LEC and signoff, so the low-power implementation stays consistent.

Q46 Where are level shifters and isolation cells placed?

Isolation cells are typically placed in the always-on or receiving domain, at the boundary of the switchable domain. They need always-on power, so a secondary power pin is routed. Level shifters are placed at the boundary, usually in the destination domain. Dual-rail level shifters need both supplies and are placed near the domain border. Placement follows the UPF location attribute (self/parent/fanout).

Placement

Q47 Stages of placement?

- Global/coarse placement: cells are placed to minimize wirelength and density, allowing overlaps (analytical/quadratic methods)
- Legalization: cells snap to rows and sites without overlap
- Detailed placement: local swaps and moves to improve wirelength and timing
- Optimization: timing-driven sizing, buffering and restructuring, with congestion repair and scan reordering

Q48 Congestion causes and fixes?

Congestion means routing demand exceeds available tracks (overflow in global routing cells). Causes: high cell density, high pin-density cells (big AOI/MUX), macro corners and narrow channels, poor macro placement, high-fanout nets, too many PG straps. Fixes: cell padding, density screens or partial blockages, spreading, macro reorientation or relocation, reducing local utilization, avoiding complex cells (dont_use), congestion-driven placement, adjusting PG strap pitch, and adding more layers.

Q49 Scan-chain reordering—what and why?

Synthesis stitches scan chains in arbitrary or alphabetical order, which leads to long zig-zag scan wires after placement. Reordering reconnects the scan chain based on physical location (using the SCANDEF file) to cut wirelength and congestion while keeping the chain length and scan-in/scan-out ports. Lockup latches are kept between clock domains.

Q50 Spare cells?

Spare cells are unused gates (NAND, NOR, INV, FF, MUX) scattered across the core with their inputs tied off. Post-tapeout functional fixes can use them in metal-only ECOs without changing the expensive base layers (FEOL masks). Today gate-array (programmable) ECO filler cells are also used.

Q51 Optimization transforms?

Pre-placement: dont_touch handling, removing assign statements and buffers from synthesis, tie cells. Post-placement transforms: gate sizing, Vt swap, buffer insertion/removal, pin swapping, cloning, logic restructuring, cell relocation, area recovery and leakage recovery.

Q52 High-fanout net synthesis (HFNS)?

Nets with very large fanout (resets, scan enable, select lines; not clocks) are buffered into a tree to meet max transition, capacitance and fanout limits. It is done in placement optimization. Clock nets are excluded because CTS handles them.

Q53 Magnet placement, bounds and regions?

Magnet placement pulls cells connected to a fixed object (a macro or port) close to it. Bounds or regions constrain a group of cells to an area: soft (guide), hard (must stay in, others may enter) or exclusive (must stay in, others must stay out). They are used to keep a hierarchy together, cut wirelength, or meet voltage area requirements.

Clock Tree Synthesis

Q54 Goals of CTS?

Deliver the clock to every sink with minimum skew, controlled insertion delay (latency), good transitions and duty cycle, and minimal power and area, while meeting max transition, capacitance and fanout limits and preserving setup and hold.

Q55 CTS inputs and constraints?

Clock definitions from SDC, the CTS spec (target skew, max latency, max transition, max capacitance, max fanout), allowed buffer/inverter cells (balanced clock cells), NDRs, routing

layers, clock exceptions (exclude, stop and float pins, ignore pins), and the clock gating structure.

Q56 Why clock inverters vs buffers?

Inverters have balanced rise/fall delays, so pulse width and duty cycle are preserved, and they are smaller (one stage vs two). A pair of inverters inserts less delay than a buffer. Special clock cells are designed with balanced Pmos/Nmos. Using an even number of inverter stages keeps polarity.

Q57 NDR for clocks—why double width/spacing?

Double width lowers resistance, which reduces RC delay and EM risk. Double spacing lowers coupling capacitance, which reduces crosstalk delta-delay and noise on the clock (clocks switch constantly and are critical). Clocks are often shielded and routed on middle-upper layers. The cost is extra routing resource.

CTS — Advanced

Q58 H-tree, X-tree, fishbone, mesh, multi-source CTS?

An H-tree is a symmetric recursive H, so all sinks are equidistant and skew is near zero, but it is hard with macros. An X-tree uses diagonals. Fishbone (spine) uses a trunk with ribs. A mesh drives a grid shorted across the whole area by many drivers. It gives very low skew and good variation tolerance but costs high power and needs SPICE-level analysis. Multi-source CTS (MSCTS) combines an H-tree to many tap points, each driving a local tree. It is a balance used in high-performance CPUs and GPUs.

Q59 Useful skew—when helpful or dangerous?

You intentionally skew clocks to borrow time: delay the capture clock of a failing setup path, taking slack from the next stage. It helps when adjacent stages have positive slack. It is dangerous because it eats hold margin, adds latency (OCV sensitivity), interacts across modes and corners, and can ripple through many paths.

Q60 OCV and CRPR/CPPR?

OCV (on-chip variation) means identical cells have different delays across the die because of process, voltage and temperature variation. STA applies derates: in a setup check the launch path is made late and the capture path early. The common clock path shared by launch and capture can't be both fast and slow at the same time. CRPR/CPPR (clock reconvergence/common path pessimism removal) adds back that artificial pessimism: credit = (late − early) delay of the common portion. A long common path means more benefit, so branch as late as possible.

Q61 Clock gating and ICG cells? Clock gating checks?

Clock gating turns off the clock to idle registers, cutting dynamic power. An ICG (integrated clock gating) cell has a latch plus an AND gate: the latch is transparent when CLK is low, so the enable is stable during the high phase and there are no glitches. STA performs clock gating setup/hold checks on the enable relative to the clock at the gate. Place ICGs near their sinks (or balance) and expect them to cost skew.

Q62 Why is the clock the highest-power net? How do you reduce clock power?

It switches every cycle (α = 1) and has huge capacitance (thousands of sinks plus tree buffers). It is often 30–50% of dynamic power. Reduce it with clock gating (multi-level), multibit flops, fewer and larger buffers (lower C), smaller trees, lower clock frequency or voltage domains, mesh avoidance, and NDRs only on the trunk.

Q63 CTS with multiple clocks sharing sinks (clock muxing)?

Define both clocks (and mark them logically exclusive with set_clock_groups -logically_exclusive if they are muxed). Balance the tree for the worst case or for each mode. Build the tree after the mux point, balancing both inputs to the mux (place the mux near the root). Use MCMM so CTS optimizes across all modes. In test mode, the scan clock usually shares the same tree.

Routing

Q64 Stages of routing?

Global routing divides the chip into GCells and assigns approximate paths, estimating congestion. Track assignment puts segments on actual tracks/layers. Detailed routing creates the exact wires and vias meeting DRCs. Search and repair fixes the remaining DRC violations iteratively. Post-route optimization then handles timing, SI and DRV.

Q65 DRC violations after routing?

Shorts, spacing, min width, min area, min enclosed area, via enclosure, end-of-line (EOL) spacing, cut spacing, min step, parallel run length, notch, open/dangling wires, coloring (double patterning) conflicts and antenna violations.

Q66 Antenna effect and fixes?

During fabrication, long metal connected to a gate collects charge from plasma etching. If the ratio of metal area to gate area exceeds a limit, the charge discharges through the thin gate oxide and damages it. Fixes: layer jumping/bridging (break the long wire by jumping to a higher layer near the gate, so the gate sees only short lower metal during etch), insert antenna diodes near the gate to safely discharge, or add buffers to split the net.

Q67 Crosstalk delay and noise; aggressor and victim?

Coupling capacitance between adjacent nets lets switching on one net (the aggressor) affect a neighbour (the victim). Crosstalk delay: opposite-direction switching slows the victim, same-direction speeds it up. This hurts setup and hold respectively. Crosstalk noise (glitch): the victim is quiet but a bump is induced on it. If it exceeds the noise margin, especially on clocks or resets, it can cause functional failure.

Q68 How do you fix crosstalk?

Increase spacing (NDR), shield the victim with VDD/VSS wires, upsize the victim driver (more holding strength), downsize the aggressor or slow its slew, insert buffers to break long parallel runs, change layers, or reroute to reduce parallel length. Timing windows also help: if aggressor and victim can't switch at the same time, the impact is filtered.

Q69 Why are top metals thicker? Which layers for clock and power?

Upper layers carry long global routes and power. Thick, wide wires have low resistance, which gives lower RC delay, lower IR drop and higher EM capability. Lower layers are thin and fine-pitch for dense local connections. Power uses the top thick layers (plus M1/M2 rails). Clocks use middle-upper layers with NDR. Long critical signals are promoted to higher layers.

Q70 Double/multi patterning (coloring)?

Below about 20nm, metal pitch is smaller than one 193nm immersion exposure can print. Layers are decomposed into 2 (LELE), 3 or more masks, or done with SADP/SAQP. Adjacent shapes closer than the same-mask spacing must get different colors. Odd cycles cause coloring conflicts. Routers must be color-aware, and same-color spacing rules apply. EUV reduces but does not eliminate this.

Q71 Redundant via insertion—why?

Single vias can fail (voids, misalignment) and cause opens. Doubling the cuts improves yield and reliability, and also reduces via resistance and EM risk. It is done post-route wherever space allows, often required as a percentage by the foundry or for DFM.

Round 3 — STA and Signoff (Deep Dive)

Static Timing Analysis

Q72 Types of timing paths?

Input port→register, register→register, register→output port, and input→output (combinational feedthrough). Also clock gating paths and asynchronous paths (recovery/removal checks on reset/set pins).

Q73 OCV vs AOCV vs POCV/SOCV (LVF)?

Flat OCV applies one derate (e.g. ±8%) to every cell, which is very pessimistic for long paths. AOCV derates by logic depth and distance: variation averages out over many stages, so deeper paths get smaller derates (table-based). POCV/SOCV models each cell delay as a mean plus sigma and combines sigma statistically (RSS) along the path. LVF (Liberty Variation Format) stores the per-arc sigma in the .lib. It is the most accurate and least pessimistic, and is required at 16nm and below.

Q74 PVT corners; worst setup/hold corners; temperature inversion?

The corners are combinations of Process (SS/TT/FF/SF/FS), Voltage (±10%) and Temperature (−40/25/125°C), plus RC extraction corners (Cworst, Cbest, RCworst, RCbest, typical). Traditionally setup is worst at SS, low V, high temperature and hold at FF, high V, low temperature. Temperature inversion: at low-VDD advanced nodes, Vt rises at low temperature more than mobility improves, so cells are slower when cold. SS/low V/−40°C can then be the worst setup corner, so both temperatures must be signed off.

Q75 What is MCMM?

Multi-corner multi-mode. The design must meet timing in every functional mode (functional, test/scan, sleep, different frequencies) across all relevant PVT/RC corners. Tools analyze and optimize all active scenarios at once, so a fix in one doesn't break another. A scenario is one mode plus one corner.

Q76 GBA vs PBA?

Graph-based analysis propagates the worst slew at each node, regardless of which path it came from. It is fast but pessimistic. Path-based analysis recomputes each path with its actual slews (and actual AOCV depth). It gives less pessimism (often tens of ps) but is slower. Use GBA for optimization and PBA for final signoff and ECO, recovering margin.

Q77 What are derates and how are they applied?

Derates are multiplicative factors applied to cell or net delays to model variation: set_timing_derate -late 1.05 -early 0.95 (optionally -cell_delay / -net_delay / -clock / -data). For setup, the launch clock and data get late derates and the capture clock gets an early derate. For hold, the reverse applies. Foundries provide recommended values per node.

Q78 Time borrowing in latches?

Latch-based designs allow data to arrive after the latch opening edge, while it is still transparent, as long as it arrives before the closing edge minus setup. The time "borrowed" is taken from the next stage. This gives the design flexibility for unbalanced stages. STA reports the borrowed time, limited by set_max_time_borrow.

Q79 What does SI-aware STA add?

It considers coupling capacitance from SPEF and the timing windows of aggressors to compute delta delay (slowdown or speedup) for each victim net, plus glitch/noise analysis. Setup uses the slow-down delta, hold uses the speed-up delta. A design clean without SI can fail with SI.

Q80 Timing DRVs and fixes?

Max transition: upsize the driver, add buffers, split the load, shorten wires. Max capacitance: buffer the net or upsize the driver, reduce fanout. Max fanout: clone the driver or buffer tree. DRVs must be fixed before timing because cell delays outside the .lib table range are extrapolated and unreliable.

Q81 Timing meets in PnR but fails in PrimeTime—why?

Miscorrelation causes: a different extractor (PnR fast RC vs StarRC/QRC signoff), different SI settings, different derates/OCV mode (GBA vs PBA, AOCV/POCV), different libs or corners, missing SDC sections, a different CRPR setting, ideal vs propagated clocks, unannotated nets, and different handling of case analysis or modes. Fix it by aligning settings, using signoff-driven ECO, and adding margins in PnR.

Q82 Setup and hold both fail on the same path—what then?

This usually means large skew or uncertainty, or a big min/max delay spread (big OCV derate, very different corners). First fix the clock: reduce the skew between launch and capture, and check for bad clock structure. Then for data: add hold delay only on the branch that fails hold (not shared with setup-critical paths), upsize on the setup side, use delay cells with low variation. Check whether a constraint is wrong (missing MCP hold setting, wrong clock relationship).

Q83 Hold fails at the fast corner but buffers break setup at the slow corner?

Find the paths where both are tight across corners. Options: use delay cells whose delay ratio between FF and SS is favorable, insert hold buffers at points where the setup-critical fanout isn't affected (endpoint side), reduce skew in the clock tree instead of padding data, use HVT cells or downsize on the hold path, use PBA to remove pessimism, use useful skew on non-critical neighbours, or as a last step reduce uncertainty margins if justified. MCMM-aware hold fixing (checking setup slack in all corners) is key.

Q84 Half-cycle paths?

Paths from a positive-edge flop to a negative-edge flop (or the reverse). Setup gets only half a period, so it is tighter. Hold gets half a period of margin, so it is easier. They often appear around clock gating, DDR interfaces, or lockup latches in scan chains.

Q85 set_input_delay, set_output_delay, set_clock_uncertainty, set_clock_latency, set_clock_transition?

- set_input_delay: external delay before data arrives at an input port, relative to a clock

- set_output_delay: external delay, including setup, required after the output port
- set_clock_uncertainty: jitter plus margin (pre-CTS also models skew); separate setup and hold values
- set_clock_latency: source latency (external) and network latency (ideal pre-CTS estimate)
- set_clock_transition: ideal clock slew used before CTS; replaced by propagated clocks after CTS

Physical Verification

Q86 DRC, LVS, ERC, antenna, density?

DRC checks that the layout obeys foundry geometric rules (width, spacing, enclosure, area). LVS compares devices and connectivity in the layout against the schematic/netlist. ERC checks electrical rules: floating gates/nets, wells not tied, shorted supplies. Antenna checks gate-damage charge ratios. Density checks verify min/max metal and poly density per window for CMP uniformity.

Q87 Common LVS errors and debug?

Shorts (two nets merged; trace the short path with highlight, often at power/signal overlap), opens (missing via or connection), device mismatch (missing or extra devices, wrong cell type), property mismatch (W/L, fin count, value), pin mismatch (missing labels, text on the wrong layer), and missing power connections or wells. Debug by fixing shorts first, using the tool's cross-probe, checking labels, then black-box handling and soft connects.

Q88 Metal fill / dummy fill—why density rules?

CMP (chemical-mechanical polishing) removes material unevenly when pattern density varies: dishing and erosion cause thickness variation. Dummy metal shapes are inserted in empty areas to keep density between min and max limits (e.g. 20–80%) in each window. Fill adds coupling capacitance, so timing-aware fill is used and signoff extraction includes the fill.

Q89 Soft-connect (well/substrate) error?

A connection made through a high-resistance layer (nwell, substrate) instead of metal, e.g. two VDD regions joined only by a well. LVS flags it as a soft connect because it can't carry current properly. Fix it by adding real metal connections or taps.

Power and Reliability Signoff

Q90 How do you fix dynamic IR drop?

Add decap near the hotspots, spread high-activity cells (padding, partial blockage), strengthen the grid locally (extra straps or vias, wider rails), stagger clock gating or switching activity, reduce simultaneous switching (skew clocks across regions), downsize drivers that aren't timing-critical, and for power switches review inrush and switch count.

Q91 Signal EM vs power EM; AC vs DC EM?

Power EM is on PG nets, where current flows in one direction (DC EM, worst case, average current). Signal EM is on signal and clock nets with bidirectional current (AC EM). It is checked against average, RMS (Joule heating) and peak limits. Signal nets partially self-heal because the current direction alternates.

Q92 Self-heating in FinFET?

The fin is thin and surrounded by oxide with poor thermal conductivity, so heat from current builds up locally. It raises device temperature, which degrades mobility and worsens EM and aging. Signoff includes self-heating-aware EM (temperature delta per device) and limits on fin/finger count per driver.

Other Signoff

Q93 Formal equivalence (LEC/Formality); when run?

LEC mathematically proves that two netlists are functionally equivalent, without vectors. It compares key points (flops, ports) and their logic cones. Run it RTL vs synthesized netlist, synthesis vs post-scan netlist, pre-PnR vs post-route netlist (after CTS, optimization and ECOs), and on every functional ECO. Low-power LEC also uses UPF.

Q94 What is an ECO? Functional vs timing; pre-mask vs metal-only?

An Engineering Change Order is a small incremental change to the design without rerunning the whole flow. A functional ECO changes logic (a bug fix). A timing ECO does sizing, buffering or hold fixes for signoff closure. A pre-mask (all-layer) ECO is done before tapeout, so any layer may change. A metal-only (post-mask) ECO uses spare or gate-array cells and changes only metal/via masks to save cost and time after silicon.

Q95 Tapeout checklist?

Clean DRC (including antenna, density, DFM), LVS, ERC; STA signoff in all MCMM scenarios (setup, hold, DRV, SI, noise, clock gating, min pulse width); IR/EM signoff; LEC clean; low-power checks (CLP/MVRC); DFT/ATPG coverage; fill inserted; GDS merged with IP and seal ring; final review of waivers; documentation. Then GDSII/OASIS is delivered to the foundry.

Round 4 — Advanced / Project / Scenario Round

The interviewer goes deep on your resume project. Answers here describe the structure a good answer should have. Replace the numbers with your own project's numbers.

Q96 Walk me through your block.

Use a structured summary: technology node and library (e.g. 7nm, 9-track), instance count (e.g. 1.2M), area and utilization, frequency per clock, number of clocks and domains, macros (e.g. 40 SRAMs), power domains and UPF features, tools used, your role (floorplan through signoff), key challenges (e.g. congestion at memory corners, hold after CTS), what you did and the measurable results (e.g. WNS from −120ps to +5ps, TNS to 0, IR drop from 9% to 4%).

Q97 Hardest timing or congestion issue and how you closed it?

Use STAR (Situation, Task, Action, Result). Example: a register-to-memory path failed by 150ps because the memory sat far from its logic. Action: re-floorplanned the memory facing the logic, used bounds to pull the logic closer, promoted nets to higher layers, applied useful skew on the memory clock, and swapped critical cells to LVT. Result: closed with +10ps and only a 3% leakage increase.

Q98 Congestion hotspots near macros after placement—debug?

Check the congestion map and overflow per layer. Find the root cause: macro pins facing away, narrow channels, cells crowding macro corners, PG blocking tracks, or high pin-density cells. Remedies: halos or partial blockages at macro corners, widen the channels, reorient the macros, cell padding, adjust PG on the congested layers, lower utilization locally, and re-run with congestion-driven placement. Verify with global route overflow.

Q99 After CTS, hold violations jumped to 10,000—what went wrong?

Likely causes: a wrong CTS skew target or bad balancing, missing set_propagated_clock (or clock uncertainty still carrying pre-CTS skew, which double-counts), the multicycle-path hold setting missing, clocks of the same family not balanced (generated and master trees, ICG levels), wrong exclusion or stop pins, or the scan path shifting. Check skew reports and whether the violations cluster per clock pair. Many may be scan paths with zero logic, which are fixed easily with lockup latches or buffers.

Q100 Timing is clean but IR drop fails in one region—options without re-floorplanning?

Add decaps or filler-decaps in the area, spread cells (padding, local density screen) via incremental placement, add local straps or vias in free tracks, swap high-power cells to lower-drive versions with slack available, distribute clock gating or rebalance activity, and recheck with vector-based analysis to remove pessimism. Then re-verify timing with IR-aware STA.

Q101 Floorplanning 50+ macros with tight area?

Study connectivity (data-flow analysis, macro grouping by hierarchy), stack and abut macros of the same type with shared channels, keep channels only where routing needs them, put macros on the periphery in arrays, align their pins toward the logic, use auto macro placement as a seed

and then refine manually, run quick trial placement and global route to evaluate congestion and timing, and iterate.

Q102 Timing budgeting across hierarchical partitions?

Split each top-level path's available time between the source block, top-level routing and the destination block, based on logic depth and the floorplan. The budgets become set_input_delay/set_output_delay on the block ports. Refine them iteratively with feedthroughs and ILM/ETM data. Interface paths are re-timed at the top with real block models.

Q103 Challenges at 5/3/2nm?

Gate-all-around transistors, backside power delivery (power rails moved to the wafer's back side, freeing front-side tracks), complex DRCs (cut rules, color rules, via pillars), larger wire RC making wires dominate delay, more variation (LVF essential), self-heating, more corners, aging and reliability, buried power rails, cell-height scaling (track count), and long runtimes.

Q104 LDE, WPE and LOD?

Layout dependent effects change transistor behavior based on the surrounding layout. WPE (well proximity effect): ions scatter off the well edge during implant, so devices near the well edge get a shifted Vt. LOD (length of diffusion, STI stress): mechanical stress from shallow trench isolation alters mobility depending on diffusion length. Others include poly spacing and OD-to-OD effects. Library cells are characterized with context, and analog layout must manage them carefully.

Q105 AI/ML-driven PnR?

Tools like Synopsys DSO.ai and Cadence Cerebrus use reinforcement learning or Bayesian search over the flow parameter space (utilization, effort settings, placement or CTS knobs) and run many trials in parallel. They often find better PPA and reduce engineer iterations. Tradeoffs are compute cost, licenses, limited explainability, and the fact that results still need verification. Engineers still own floorplan intent and debugging.

Q106 PPA tradeoffs?

Performance, power and area pull against each other: faster cells (LVT, upsizing) increase leakage and area, lower utilization eases timing but costs area, and clock gating saves power but complicates CTS and timing. Prioritize based on the product (mobile focuses on power, server on performance, IoT on area and cost). Use multi-Vt, multibit flops, power gating and selective optimization.

Q107 3D-IC, chiplets, TSV, interposer?

Chiplets split a system into smaller dies (possibly different nodes), joined on a package or 2.5D silicon interposer (e.g. CoWoS) with high-density micro-bumps. 3D-IC stacks dies vertically with TSVs (through-silicon vias) or hybrid bonding. Challenges include die-to-die timing, thermal, power delivery, inter-die DRC/LVS, testing (known-good-die), and co-design tools.

Q108 Low-power techniques you implemented?

Clock gating (RTL and tool-inserted, multi-level), multi-Vt optimization, power gating with switches, isolation and retention, multi-voltage domains with level shifters, DVFS (dynamic voltage-frequency scaling at runtime), body biasing (forward for speed, reverse for leakage; more effective in FD-SOI), multibit flops and memory light-sleep modes. Be ready to explain the UPF.

Q109 Tool commands for timing, congestion and QoR?

Innovus: report_timing, timeDesign -preCTS/-postCTS/-postRoute, reportCongestion, report_qor / summaryReport, verify_drc, ccopt_design. ICC2/Fusion Compiler: report_timing, report_qor, report_congestion, report_constraints -all_violators, check_routes, clock_opt, route_opt. PrimeTime: report_timing -delay_type max/min -pba_mode path, report_constraint -all_violators, report_global_timing, fix_eco_timing.

Round 5 — Scripting and Automation

Q110 TCL: report the top 10 worst paths and the cells on them.

```
set paths [get_timing_paths -max_paths 10 -nworst 1 -delay_type max]
foreach_in_collection p $paths {
  set sp [get_object_name [get_attribute $p startpoint]]
  set ep [get_object_name [get_attribute $p endpoint]]
  puts "Slack: [get_attribute $p slack]  $sp -> $ep"
  foreach_in_collection pt [get_attribute $p points] {
    set obj [get_attribute $pt object]
    puts "   [get_object_name $obj]  ([get_attribute [get_cells -of $obj] ref_name])"
  }
}
```

Q111 TCL: find all nets with fanout > 50 and their drivers.

```
foreach_in_collection n [get_nets -hier *] {
  set loads [get_pins -quiet -leaf -of $n -filter "direction==in"]
  if {[sizeof_collection $loads] > 50} {
    set drv [get_pins -quiet -leaf -of $n -filter "direction==out"]
    puts "[get_object_name $n] fanout=[sizeof_collection $loads] driver=[get_object_name
[get_cells -of $drv]]"
  }
}
```

Many tools also support get_nets -filter "fanout > 50" directly, depending on the attribute name in that tool.

Q112 Explain foreach_in_collection, get_cells -filter, get_attribute, lsort, regexp, dict.

- foreach_in_collection: iterates over EDA tool collections, which are object handles rather than TCL lists
- get_cells -filter "ref_name =~ *LVT*": selects objects matching an attribute expression
- get_attribute obj attr: reads properties such as slack, area or ref_name
- lsort: sorts a list (-real, -decreasing, -index)
- regexp: regular-expression match and capture, e.g. regexp {slack \(VIOLATED\)\s+(-?[\d.]+)} $line -> s
- dict: key–value store, e.g. dict incr counts $ref to count cells per type

Q113 Python: parse a timing report into CSV.

```
import re, csv
rows, cur = [], {}
for line in open("timing.rpt"):
    if m := re.match(r"\s*Startpoint:\s+(\S+)", line): cur = {"start": m.group(1)}
    elif m := re.match(r"\s*Endpoint:\s+(\S+)", line): cur["end"] = m.group(1)
    elif m := re.search(r"slack\s+\((?:MET|VIOLATED)\)\s+(-?[\d.]+)", line):
        cur["slack"] = float(m.group(1)); rows.append(cur)
with open("paths.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["start", "end", "slack"])
    w.writeheader(); w.writerows(sorted(rows, key=lambda r: r["slack"]))
```

Q114 Shell: extract violations from a log.

```
grep -c "VIOLATED" timing.rpt                          # count violations
grep "slack (VIOLATED)" timing.rpt | awk '{print $NF}' | sort -n | head   # worst slacks
grep -E "^\*\*ERROR|^ERROR" run.log | sort | uniq -c | sort -rn             # error summary
sed -n '/Startpoint/,/slack/p' timing.rpt > first_paths.txt                # extract blocks
```

Q115 Makefile for automating a PnR flow?

Define one target per stage with dependencies on the previous stage's database, e.g. floorplan.db: syn.v → place.db → cts.db → route.db → signoff. Each recipe runs the tool in batch with a stage TCL script and writes a log. Make reruns only the out-of-date stages and allows parallel corners (make -j) or LSF submission (bsub). Add targets for clean, reports and a QoR summary.

Q116 Automating regression and QoR tracking?

Run the flow nightly or per change (cron/Jenkins/LSF), parse reports (WNS, TNS, violations, utilization, congestion overflow, DRC count, IR, runtime) into CSV or a database, compare against a baseline, flag regressions, and publish a dashboard or email. Keep the flow scripts in git with tagged recipes.

Round 6 — Managerial and HR

Q117 Why physical design? Why this company?

Connect your interest to the work: turning a netlist into real silicon, solving multi-variable PPA puzzles, and measurable impact on tapeouts. Mention the company's products, nodes and technology (e.g. its AI accelerators, mobile SoCs or EDA tools) and how your skills fit the specific team. Be genuine and specific.

Q118 Handling a deadline crunch during tapeout?

Use STAR: describe the situation (e.g. a late netlist drop two weeks before tapeout), how you prioritized (triaged violations by impact, automated ECO loops, parallelized corners), how you communicated risks early to leads, and the outcome (taped out on time with zero signoff waivers). Show calm ownership, not heroics alone.

Q119 Disagreement with synthesis/front-end over constraints?

Show data-driven collaboration: you found an unrealistic constraint (a missing multicycle path, an over-tight IO delay), shared timing reports and path analysis, proposed options, involved the architect to confirm intent, and agreed on the fix. Emphasize respect and a shared goal.

Q120 Prioritizing multiple blocks?

Rank by tapeout criticality, dependencies (the top level needs abstracts first), and risk (blocks furthest from closure). Use trackers, automate repetitive runs, run compute-heavy jobs overnight, and communicate status and risks regularly with the lead.

Q121 Notice period, salary, relocation?

Be honest and research market ranges for your experience and location. Give a range based on total compensation, show flexibility, and state relocation preferences clearly.

Related Roles — Focus Areas and Sample Q&A

Role	Extra emphasis areas
STA Engineer	SDC writing/debug, MCMM, OCV/POCV, CRPR, PBA, ECO generation (PT-ECO, Tweaker), CDC constraints, closure strategy
Physical Verification	DRC/LVS/ERC decks, SVRF/TVF scripting, Calibre debug, DFM, fill/density, ESD and latch-up
Synthesis Engineer	DC/Genus flow, constraint QA, retiming, dont_touch/dont_use, LEC, multibit, physical-aware synthesis
Power/IR/EM	Grid design, vectorless vs vector IR, RedHawk/Voltus, decap planning, EM rules, ESD
DFT Engineer	Scan insertion, ATPG, stuck-at/transition faults, MBIST, compression, scan reordering
CAD/Flow Engineer	TCL/Python mastery, flow development, Make/LSF, regression systems, tool integration
Analog Layout	Matching, common centroid, interdigitation, guard rings, shielding, parasitics, LDE

Q122 (STA) How do you constrain a CDC path between asynchronous clocks?

Declare the clocks asynchronous with set_clock_groups -asynchronous -group {clkA} -group {clkB}, or use set_false_path between them. For multi-bit gray-code buses or FIFO pointers, limit skew with set_max_delay -datapath_only (bus skew) so the bits stay coherent. Synchronizer correctness itself is verified with CDC tools (SpyGlass/Questa CDC), not STA.

Q123 (Synthesis) What is retiming?

Retiming moves registers across combinational logic to balance stage delays without changing functionality (the cycle-level behavior at the outputs). It improves frequency but changes register names, which complicates LEC (needs special setup) and debug.

Q124 (DFT) Stuck-at vs transition fault?

A stuck-at fault models a node permanently at 0 or 1 and is tested with slow-speed scan. A transition fault models a node that is too slow to rise or fall and needs at-speed testing (launch-on-capture or launch-on-shift) using the functional clock. Both use scan chains to control and observe internal flops.

Q125 (PV) What is ESD protection and how is it checked?

Electrostatic discharge events (HBM, CDM) send large currents into the pins. I/O pads have clamp diodes and power clamps to divert the current safely. PV checks the ESD path resistance, the presence of clamps, the spacing of protection devices, and current density along discharge paths (Calibre PERC).

Q126 (Analog layout) Why common-centroid layout?

Process gradients (oxide thickness, doping) vary linearly across the die. Arranging matched devices symmetrically around a common center (e.g. ABBA / BAAB) cancels first-order gradients,

improving matching for differential pairs and current mirrors. Dummy devices at the edges handle etch effects.

Preparation Tips

Freshers: focus on Rounds 0–2. Master CMOS, setup/hold equations and the full PnR flow, and be able to derive things on a whiteboard.

2–5 years: expect deep Round 2–3 questions and scenario questions based on your own project. Know your design's numbers.

Senior/Lead: Rounds 3–4 dominate, covering methodology, hierarchical flows, advanced nodes, mentoring and schedule ownership.

Recommended reading: "Static Timing Analysis for Nanometer Designs" (Bhasker and Chadha), "CMOS VLSI Design" (Weste and Harris), "Physical Design Essentials" (Golshan), "VLSI Physical Design" (Kahng et al.), plus tool user guides and the VLSI Expert / Team VLSI channels.